TAJ TRUST FOUNDATION
House 45, Motijheel Commercial Area
+(880) 197xxxxxx
help@tajtrust.com

Vulnerability Assessment & Penetration Testing Report on

Altoro Mutual, Inc.

VAPT • TTF • ALTORO

Security Assessment Conducted by

TTF

Offensive Security & Reconnaissance Unit

⚖ SECTION - SCOPE DECLARATION, ETHICS, AND LEGAL NOTICE

⚠ IMPORTANT — Legal & Ethical Compliance Statement

- This Vulnerability Assessment and Penetration Testing (VAPT) engagement was carried out only within the client-agreed approved parameters, timeframes, and IP ranges.
- High-risk exploits and intrusive testing methodologies were carefully controlled to minimize the risk of operational disruption, data loss, or service downtime.
- Every vulnerability listed in this report was found using legal testing methods that were carefully applied to targets that were specifically included in the active scope.

A General Overview

Altoro contracted Taj Trust Foundation to do a structured Vulnerability Assessment and Penetration Testing (VAPT) engagement against their chosen digital applications and infrastructure. This engagement's goals are to find weaknesses in security, assess how well the defenses in place are working, and recreate actual attack methods that a skilled threat actor may use to obtain unauthorized access or interfere with operations.

VAPT involves active analysis, scanning, and controlled exploitation of vulnerabilities found within the specified scope, in contrast to passive reconnaissance. The results represent a thorough assessment of Altoro's current level of security, offering practical technical knowledge and repair suggestions to actively enhance defenses prior to a violent assault.

≡ SECTION — TABLE OF CONTENTS

• **Introduction**	**4**
◦ Objective of the assessment	4
◦ Scope of Assessment	4
◦ Out of Scope	5
◦ Assumptions & limitation	5
• **Pentest Details**	**6**
◦ Assessment Methodology	6
◦ Testing Framwork	6
◦ Risk Scoring Methodology	7
◦ Validation Process	7
◦ Tools Utilized	8
◦ Summary of Vulnerability Findings	9
• **Vulnerability Details**	**10**
◦ Cross-Site Scripting (DOM Based)	11
◦ Cross-Site Scripting (Reflected)	14
◦ HTML Injection	18
◦ Clickjacking: X-Frame Injection	21
◦ SQL Injectiion	24
◦ Brute Force	28
◦ Absence Of Anti-CSRF Tokens	30

Introduction

Objective of the Assessment

The primary objective of this Vulnerability Assessment and Penetration Testing (VAPT) engagement was to evaluate the security posture of the Altoro Mutual Bank web application. The assessment aimed to identify, validate, and prioritize vulnerabilities that could be exploited by an attacker to compromise confidentiality, integrity, or availability. Specifically, this report details findings related to OWASP Top 10 vulnerabilities, authentication flaws, session management issues, and server-side misconfigurations.

Scope of the Assessment

Type	Details
Target	http://altoro.testfire.net/
Duration	3 Business Days Start Date: 28 June 2026, End Date: 30 June 2026
In-Scope	Web application, REST API endpoints (/api/v1/*), authentication & session management
Testing Approach	Grey-box — authenticated (standard user + admin) and unauthenticated testing

- Key Areas Tested:
  - HTTP Security Headers (CSP, HSTS, X-Frame-Options)
  - SSL/TLS Configuration
  - Common OWASP Top 10 Vulnerabilities (XSS, SQLi, CSRF)
  - Server Version Disclosure

Out of Scope

1. Underlying infrastructure (OS, network devices)
2. Third-party integrations
3. DNS configuration
4. Social engineering
5. Denial-of-service attacks

Assumptions & Limitation

Rate limitation: Tests were carried out with pauses to prevent automatic IP bans; it was presumed that rate limitation was in effect.

Database: Exposure through an API endpoint was the only way to evaluate direct database access.

Performance Impact: To reduce the impact on demo users, testing was done at off-peak times.

Scope Boundary: This test did not include mobile app endpoints unless they shared the same API backend.

Pentest Details

Assessment Methodology

our team used a hybrid methodology that combined Automated Scanning and Manual Expert Analysis to ensure comprehensive coverage of the attack surface of the Altoro Mutual website.

The testing lifecycle followed a structured four-phase approach:

1. Reconnaissance & Information Gathering
2. Vulnerability Analysis
3. Exploitation (Penetration Testing)
4. Reporting & Validation

Testing Framework

The assessment was conducted in accordance with the following industry standards:

- OWASP Top 10:2025 – Used as the primary vulnerability classification framework
- OWASP Testing Guide v4.2 – Guided manual testing procedures
- CVSS v3.1 – Used for vulnerability severity scoring

Risk Scoring Methodology

- Vulnerabilities are scored using the Common Vulnerability Scoring System (CVSS v4.0) and mapped to the following risk categories:
  - Critical (CVSS 9.0–10.0): Immediate remediation required
  - High (CVSS 7.0–8.9): Prompt remediation recommended
  - Medium (CVSS 4.0–6.9): Remediate within the next sprint cycle
  - Low (CVSS 0.1–3.9): Remediate during scheduled maintenance
  - Informational: No immediate action required; consider for hardening

Validation Process

All vulnerabilities were validated using a Three-Tier Verification Model:

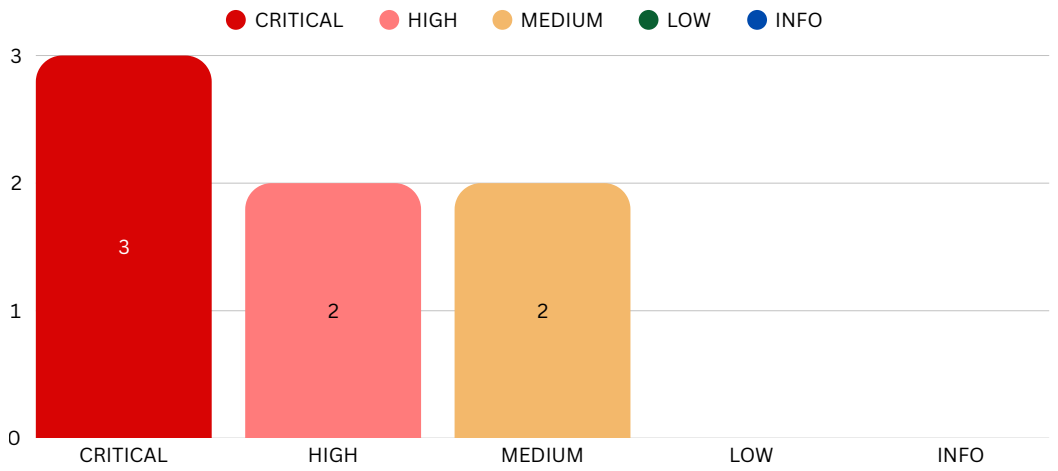
- Automated Detection: Initial flagging via scanners.
- Manual Confirmation: Analyst manually triggers the exploit to confirm it is not a false positive.
- Impact Proof: Demonstration of business impact (e.g., data exfiltration or admin access).

Tools Utilized

Tool Name	Category	Primary Purpose / Usage
Burp Suite Professional	Interception Proxy	Manual traffic manipulation, API testing, and web application fuzzing.
Acunetix & Nessus	Vulnerability Scanner	Industry-standard automated scanners utilized to identify vulnerabilities
OWASP ZAP	Web Scanner	Automated baseline vulnerability scanning and spidering.
SQLMap	Database Exploitation	Automated SQL injection detection and exploitation
Nmap	Network Discovery	Port scanning, service banner grabbing, and network mapping.
Dirsearch & Gobuster	Directory Brute-forcing	Discovering hidden files, directories, and unlinked endpoints on the server.
Nikto	Web Server Scanner	Identifying outdated server software, misconfigurations, and known vulnerabilities.
Browser DevTools	Client-Side Analysis	Inspecting DOM elements, session storage, cookies, and analyzing JavaScript execution.

Summary of Vulnerability Findings

No.	Title	Severity	Risk Score	Status	Link
01	Cross-Site Scripting (DOM Based)	HIGH	8.5	Solved	Open
02	Cross-Site Scripting (Reflected)	CRITICAL	9.3	Solved	Open
03	HTML Injection	CRITICAL	9.1	Solved	Open
04	Clickjacking:X-Frame Injection	HIGH	8.6	Solved	Open
05	SQL Injection	CRITICAL	9.8	Solved	Open
06	Brute Force	MEDIUM	6.5	Solved	Open
07	Absence Of Anti-CSRF Tokens	MEDIUM	6.5	Solved	Open



Vulnerability Details



Cross-Site Scripting (DOM Based)

XSS that allows attackers to inject malicious scripts into web pages viewed by other users	
Severity	HIGH
Affected URL	http://testfire.net/search.jsp
Risk Score	8.5/10
CWE	Improper Input Neutralization During Web Page Creation
Labels	HIPAA, PCI DSS, ISO 27001, GDPR, SOC 2, SOC 2 - Security, OWASP 2021, OWASP 2021 - A03 - Injection, XSS

Description:

During the security assessment, a DOM-based Cross-Site Scripting (XSS) vulnerability was identified. The application was found to insecurely process user-controlled input and sink it directly into the Document Object Model (DOM) environment without adequate sanitization, validation, or context-aware encoding. Specifically, the application was susceptible to a polyglot XSS payload executed via the URL/input parameter. The following payload was successfully executed

```
#jaVasCript:/*-/*`/*\`/*'/*"/**/(/* */oNcliCk=alert(5397)
)//%0D%0A%0d%0a//</stYle/</titLe/</teXtarEa/</scRipt/--!>\x3csV
g/<sVg/oNloAd=alert(5397)//>\x3e
```

This complex payload uses mixed casing, comment variations (/* ... */), encoding tricks (\x3c), and multiple HTML closing tags (such as </script>, </textarea>, </style>) to bypass weak client-side or server-side Web Application Firewall (WAF) filters. Once executed, it forces the DOM to dynamically render and execute malicious JavaScript (in this case, triggering alert(5397)) within the context of the user's browser session.

Impact:

DOM-based Cross-Site Scripting attacks execute malicious code entirely within the victim's browser context. Because the payload is executed via the DOM, server-side logs may never see the exploit traffic, making detection difficult. Adversaries utilize XSS for the following purposes:

- **Session Hijacking and Credential Theft:** To steal active session cookies, JSON Web Tokens (JWTs), or local storage data, allowing attackers to completely impersonate the victim's account without knowing their password.
- **Defacement and Content Manipulation:** To dynamically alter the visible content of the website, insert fraudulent login forms (phishing), or mislead users into performing unintended actions.
- **Data Exfiltration:** To capture sensitive information entered by the user in real-time (keylogging) or silently scrape data accessible within the application UI.
- **Forced Actions (CSRF Amplification):** To force the victim's browser to execute unauthorized actions on the application on their behalf, such as altering account settings, updating email addresses, or initiating transactions.

Recommendations:

To effectively mitigate DOM-based Cross-Site Scripting (XSS) vulnerabilities, the application must ensure that all user-controlled data is treated as untrusted and safely handled before being rendered in the Document Object Model (DOM). Implement the following remediation strategies:

1. Use Safe DOM Sinks:

Avoid using dangerous JavaScript sinks that dynamically execute or evaluate code. Replace unsafe methods with secure alternatives that treat data strictly as text, preventing the browser from parsing HTML or executing scripts:
Instead of: element.innerHTML, document.write(), or eval()
Use: element.textContent or element.innerText

2. Context-Aware Output Encoding:

If user input absolutely must be rendered as HTML, it must pass through a strict context-aware encoder.

Utilize well-maintained, proven sanitization libraries such as DOMPurify to strip out malicious HTML tags, JavaScript events (like onLoad or onClick), and URI schemes (like javascript:) before the input reaches the DOM.

3. Implement a Robust Content Security Policy (CSP):

A strong CSP provides a vital defense-in-depth layer. If a vulnerability is found, CSP restricts what the browser is allowed to execute.

Deploy a strict CSP HTTP response header (e.g., Content-Security-Policy: default-src 'self'; script-src 'self';).

Avoid the use of 'unsafe-inline' and 'unsafe-eval' directives within your script sources, as they effectively neutralize CSP protection against inline XSS payloads.

4. Input Validation and URL Whitelisting:

If the input is expected to be a URL, strictly validate it against an allowlist of approved protocols (e.g., forcing http:// or https://).

Reject or strip inputs utilizing the javascript: pseudo-protocol or data URIs to neutralize polyglot delivery attempts.

Proof of Concept (POC):

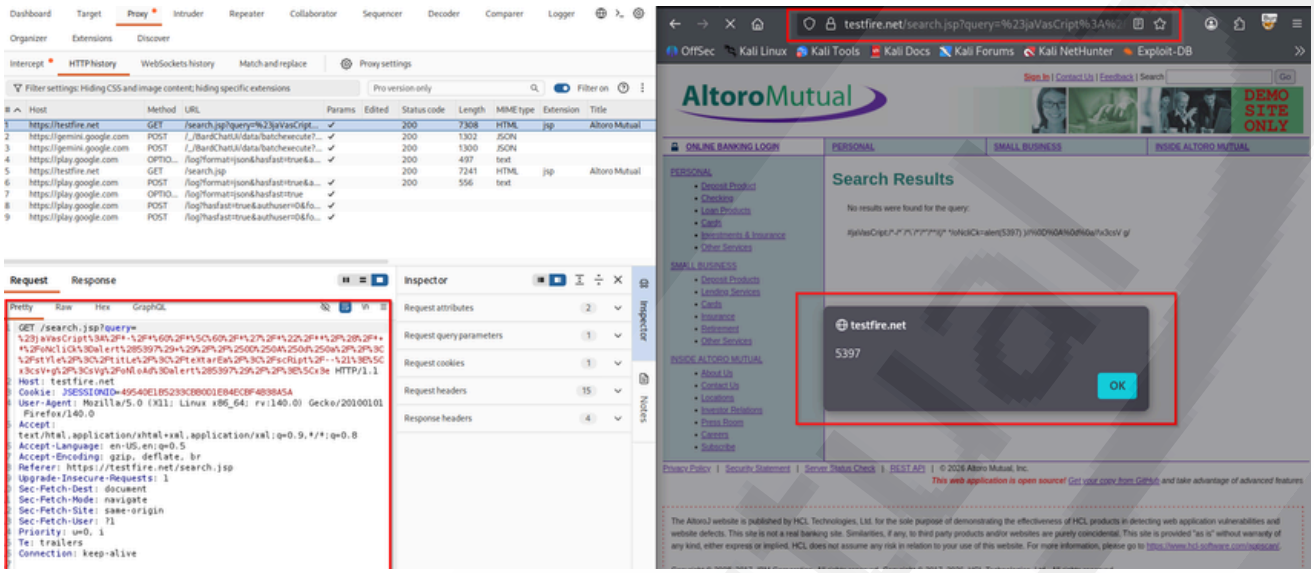
Go to the page: Open the target webpage in your browser.

Enter the payload: Paste the following code into the vulnerable input field or append it to the URL parameter:

#jaVasCript:/*-/*`/*\`/*'/*"/**/(/*
*/oNcliCk=alert(5397))//%0D%0A%0d%0a//</stYle/</titLe/</teXtarEa/</scRipt/
--!>\x3csVg/<sVg/oNloAd=alert(5397)//>\x3e

Execute: Submit the page or refresh the URL

Result: An alert box displaying 5397 will pop up on the screen, proving that arbitrary code can run.



Cross-Site Scripting (Reflected)

Reflected XSS is a attack where malicious JavaScript code is injected into a web application

Severity	CRITICAL
Affected URL	http://testfire.net/feedback.jsp
Risk Score	9.3/10
CWE	Improper Input Neutralization During Web Page Creation
Labels	HIPAA, PCI DSS, ISO 27001, GDPR, SOC 2, SOC 2 - Security, OWASP 2021, OWASP 2021 - A03 - Injection, XSS

Description:

Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.

Impact:

Successful exploitation allows an attacker to execute arbitrary JavaScript in the victim's browser session on the affected domain. This leads to:

Session Hijacking: Steal cookies/tokens → account takeover
Data Theft: Exfiltrate sensitive page content, CSRF tokens, local storage

Phishing: Render fake login forms or malicious overlays
Defacement: Modify page content visible to the victim

Malware Delivery: Load secondary payloads (BeEF, stagers)
Lateral Movement: Pivot to internal applications via the victim's browser

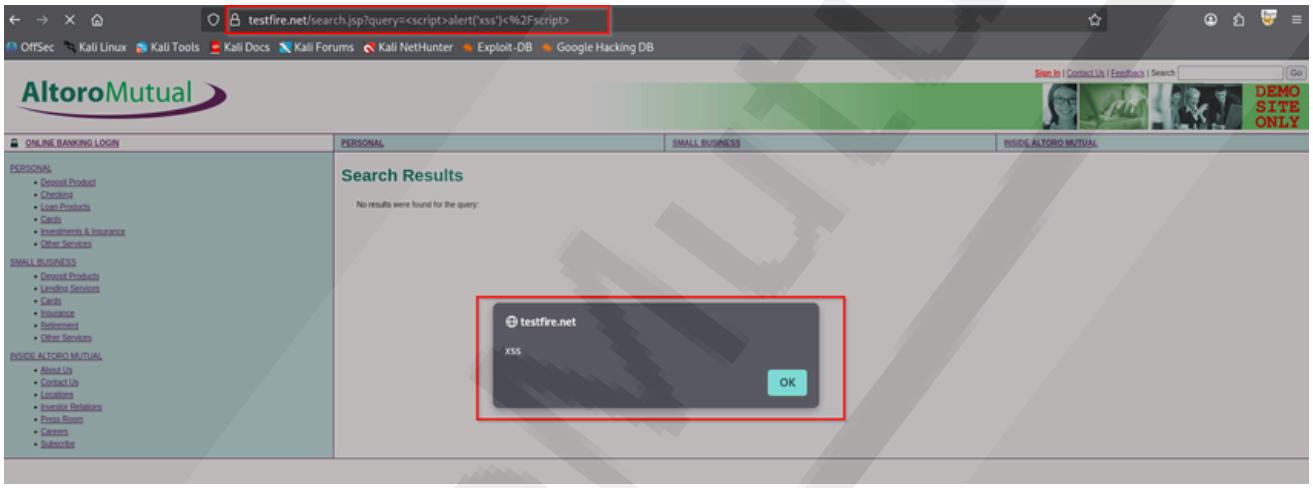
The attack requires only that the victim clicks a crafted link (UI:R). No authentication is needed (PR:N), and the impact is amplified when the vulnerability exists on a high-privilege page (admin panel, user profile management, etc.).

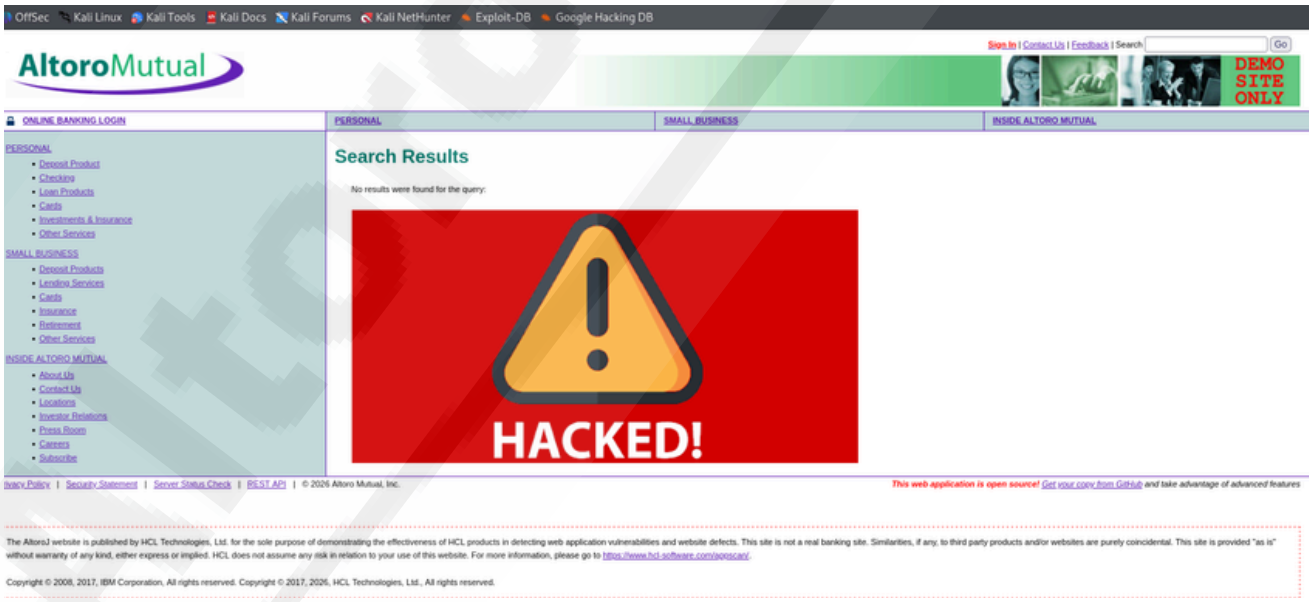
Proof of Concept (POC):

Using Search Bar to find XSS Vulnerability

Payloads:

1. <script>alert('xss')</script>
2. <svg onload=confirm('XSS')>
3. <img src=x onerror=confirm('XSS')>





Recommendations:

Preventing cross-site scripting is trivial in some cases but can be much harder depending on the complexity of the application and the ways it handles user-controllable data.

• In general, effectively preventing XSS vulnerabilities is likely to involve a combination of the following measures:
• Filter input on arrival.
• At the point where user input is received, filter as strictly as possible based on what is expected or valid input.
• Encode data on output.
• At the point where user-controllable data is output in HTTP responses, encode the output to prevent it from being interpreted as active content. Depending on the output context, this might require applying combinations of HTML, URL, JavaScript, and CSS encoding.
• Use appropriate response headers.

HTML Injection

HTML Injection leads to Defacement, Phishing Attack and Content Spoofing	
Severity	CRITICAL
Affected URL	http://testfire.net/feedback.jsp
Risk Score	9.1/10
CWE	Improper Input Neutralization During Web Page Creation
Labels	HIPAA, PCI DSS, ISO 27001, GDPR, SOC 2, SOC 2 - Security, OWASP 2021, OWASP 2021 - A03 - Injection, XSS

Description:

HTML Injection occurs when an attacker can inject malicious HTML code into a web application's output without proper encoding or escaping. In this demo website, we identified that user-supplied input was reflected directly into the HTML body of the response. This allows an attacker to alter the visual presentation of the page, hide content, or redirect users by injecting elements like headings, images, or iframes.

Impact:

Session Hijacking: Attackers can steal session cookies (document.cookie) and impersonate users.

Data Theft: Sensitive data displayed on the page (e.g., emails, personal info) can be exfiltrated to an attacker's server.

Defacement: The entire page content can be replaced with malicious content.

Phishing: Fake login forms can be injected into the page to steal credentials.

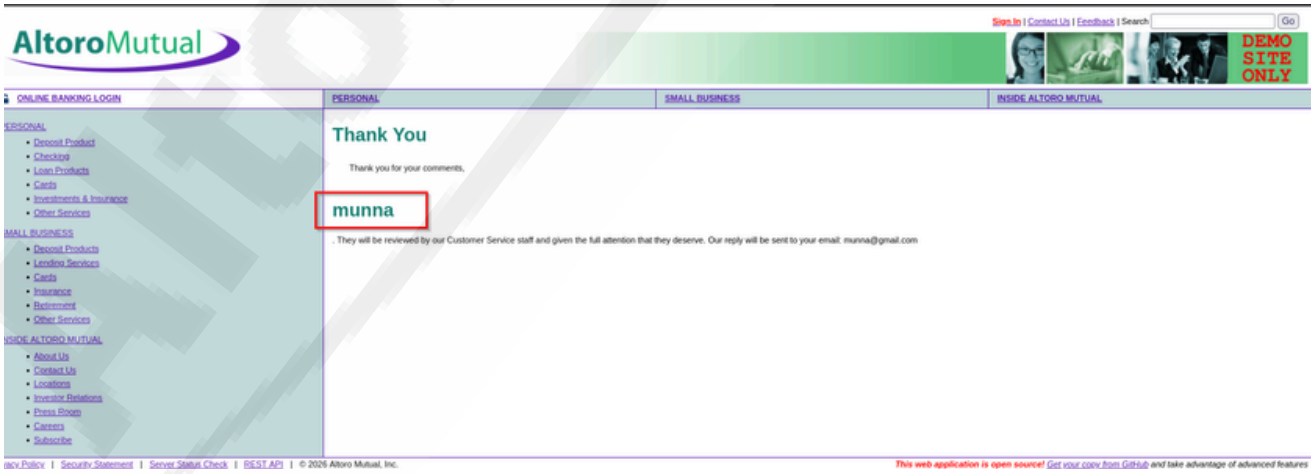
Proof of Concept (POC):

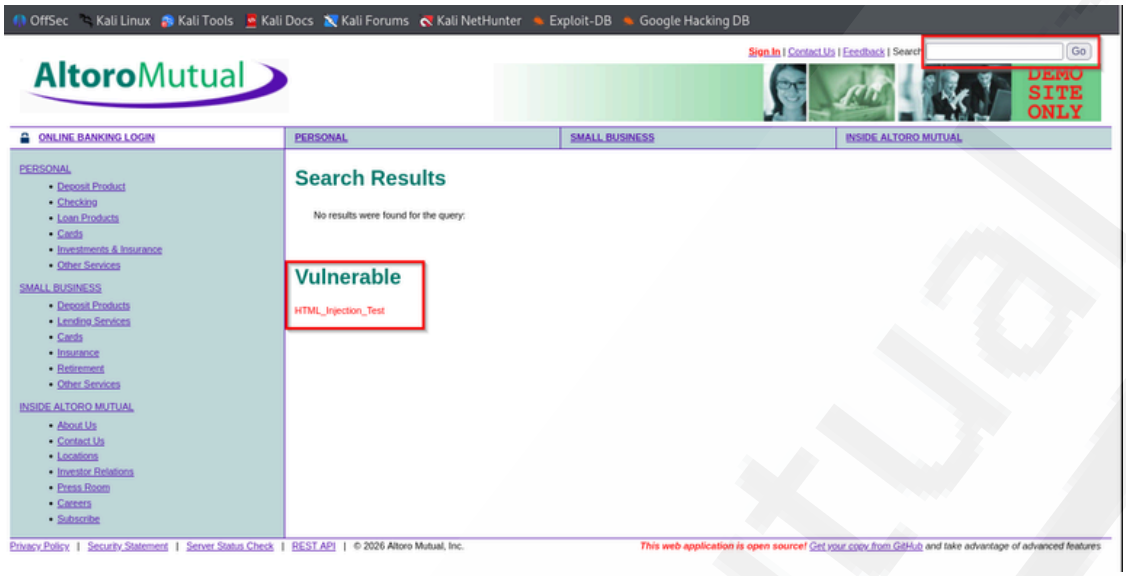
Affected URL:

1. http://testfire.net/feedback.jsp
2. http://testfire.net/login.jsp

Using Search Bar to find HTML Injection Payloads:

<h1>Vulnerable</h1>HTML_Injection_Test </h1>
<h1>Injected</h1><img src=x onerror=alert(5397)>





Recommendations:

- **Implement Context-Aware Output Encoding:** Ensure that all user-supplied data is HTML-entity encoded before being rendered on the page. Convert special characters into their safe HTML entities (e.g., convert < to <, > to >, and " to ").
- **Input Validation**: Restrict user inputs to an allowlist of expected characters (such as alphanumeric only) wherever possible. Reject any input containing structural HTML characters.
- **Use Safe Templating:** Utilize modern web frameworks or templating engines (like React, Angular, or Jinja) that automatically encode output text by default.

Clickjacking: X-Frame Injection

misconfiguration allows an attacker to trick a user into clicking invisible interface elements

Severity	HIGH
Affected URL	http://testfire.net/feedback.jsp
Risk Score	8.6/10
CWE	Improper Input Neutralization During Web Page Creation
Labels	HIPAA, PCI DSS, ISO 27001, GDPR, SOC 2, SOC 2 - Security, OWASP 2021, OWASP 2021 - A03 - Injection, XSS

Description:

Clickjacking (short for "UI Redress Attack") occurs when a web application does not prevent itself from being displayed within an HTML <iframe> on another website. In the Altoro Mutual demo, we identified that the login page (/login) and potentially other internal pages do not include the X-Frame-Options HTTP response header

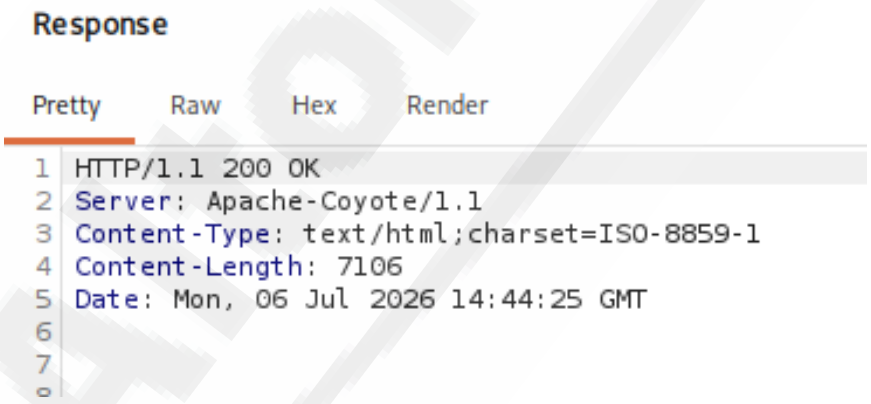
Impact:

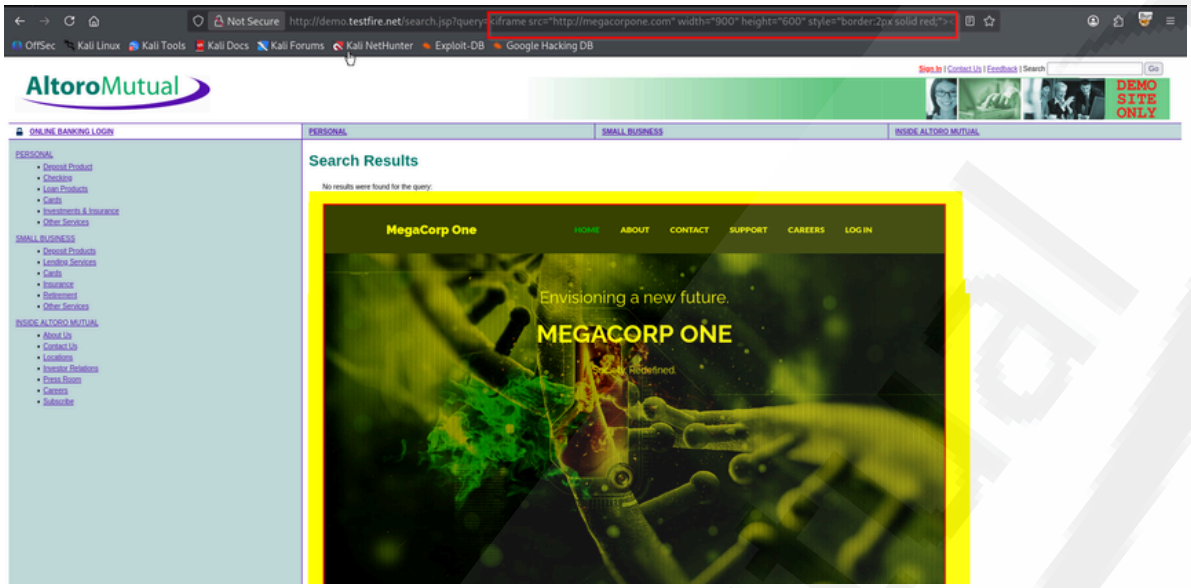
- **UI Redressing:** An attacker can trick users into clicking buttons or links on the Altoro interface without them realizing it.
- **Credential Harvesting:** The most common use case is embedding the login form. When the user enters their username and password, they are submitting them to the Altoro backend, but the attacker can capture the session cookie or redirect the user after login.
- **Phishing Enhancement:** It makes phishing attacks more convincing because the victim sees the real Altoro UI (with correct logos, styles, etc.) rather than a fake clone, reducing suspicion.
- **Unauthorized Actions:** If the user is already logged in, an attacker could frame the "Transfer Funds" page and trick the user into clicking "Submit," transferring money without their full awareness.

Proof of Concept (POC):

Check Headers: Using a tool like Burp Suite, inspect the HTTP response headers

Observation: The header X-Frame-Options is either missing or set to SAMEORIGIN (which allows iframes from the same domain, but often fails to block external domains depending on the browser/version).





Use this payload to make the iframe:

<iframe src="http://megacorpone.com" width="900" height="600" style="border:2px solid red;"></iframe>

Recommendations:

To mitigate Clickjacking, the development team should implement one of the following HTTP response headers in the Altoro application configuration:

Option A: X-Frame-Options

Set this header to DENY if the page should never be framed by any site, or SAMEORIGIN if it can only be framed by pages on the same domain.

Header: X-Frame-Options: DENY

Effect: Prevents the page from being displayed in an iframe on any other website.

Option B: Content-Security-Policy (CSP) (Modern Best Practice)

Use the frame-ancestors directive. This is more granular and allows you to specify exactly which domains can frame your content.

Header: Content-Security-Policy: frame-ancestors 'self';

Effect: Allows framing only from the same origin. To allow multiple specific domains, use: frame-ancestors 'self' https://trusted-partner.com;

SQL Injection

SQL Injection leads to unauthorized access, theft or deletion of sensitive database data	
Severity	CRITICAL
Affected URL	http://testfire.net/login.jsp
Risk Score	9.8/10
CWE	Improper Neutralization of Special Elements used in an SQL Command
Labels	OWASP Top 10: A03:2021, CAPEC-66, ATT&CK Matrix: T1190, CISA KEV

Description:

SQL Injection (SQLi) occurs when an web application constructs SQL queries using user-supplied input without proper sanitization or parameterization. In the Altoro Mutual demo, we identified that the backend database directly concatenates user input into SQL statements.

Specifically, the login endpoint (or search field) accepts unvalidated input. When a specific payload is entered, it alters the logic of the underlying SQL query. For example, entering ' OR '1'='1 in the username field can close the original string, add an OR condition that is always true, and bypass authentication checks, allowing the attacker to log in as any user (often the first admin user found) without knowing the password.

Impact:

Authentication Bypass: Attackers can log in as any user, including administrators, by manipulating the SQL query logic.

Data Exfiltration: Using Union-based injection, an attacker can extract sensitive data from other tables (e.g., users, accounts, transactions), including usernames, hashed passwords, and account balances.

Data Modification: Depending on the database permissions, an attacker may be able to update or delete records (e.g., changing their own balance).

Database Enumeration: Attackers can discover the database schema, table names, and column structures.

Remote Code Execution (RCE): In some configurations (like MySQL with INTO OUTFILE or MSSQL with xp_cmdshell), SQLi can lead to full server compromise.

Proof of Concept (POC):

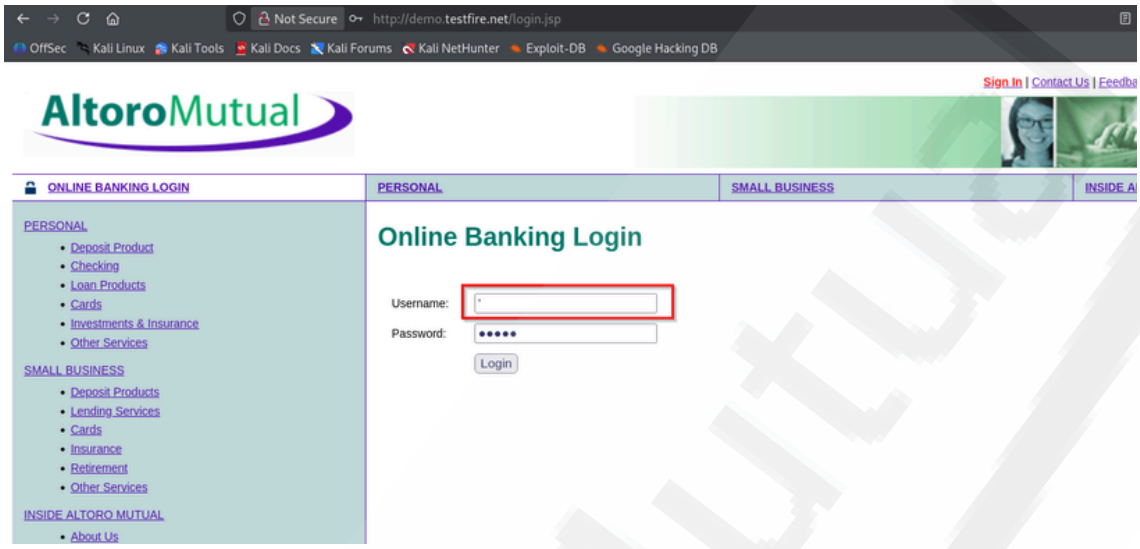
Affected URL:
http://testfire.net/login.jsp

Payload:

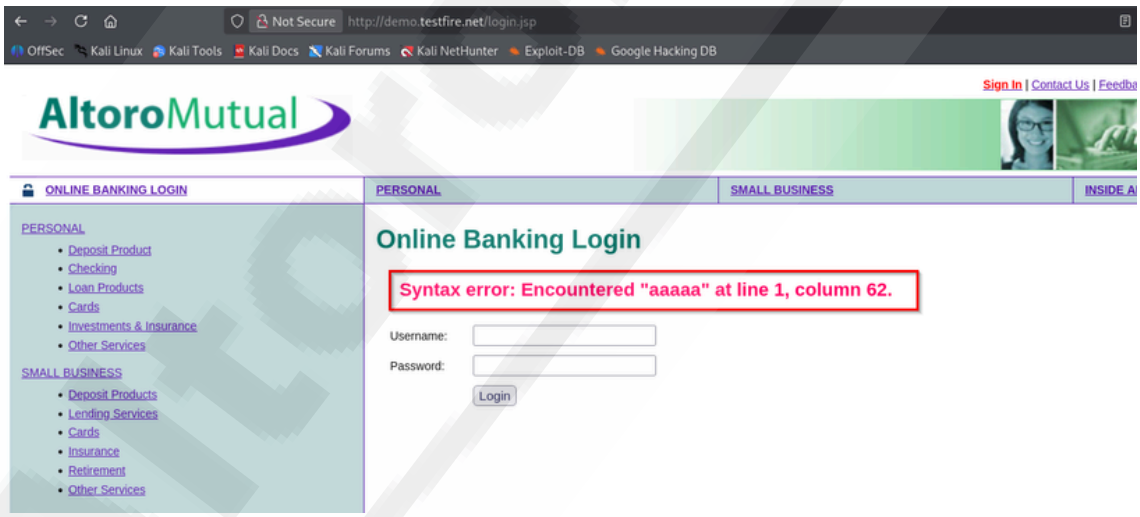
```
OR '1'='1' --
```

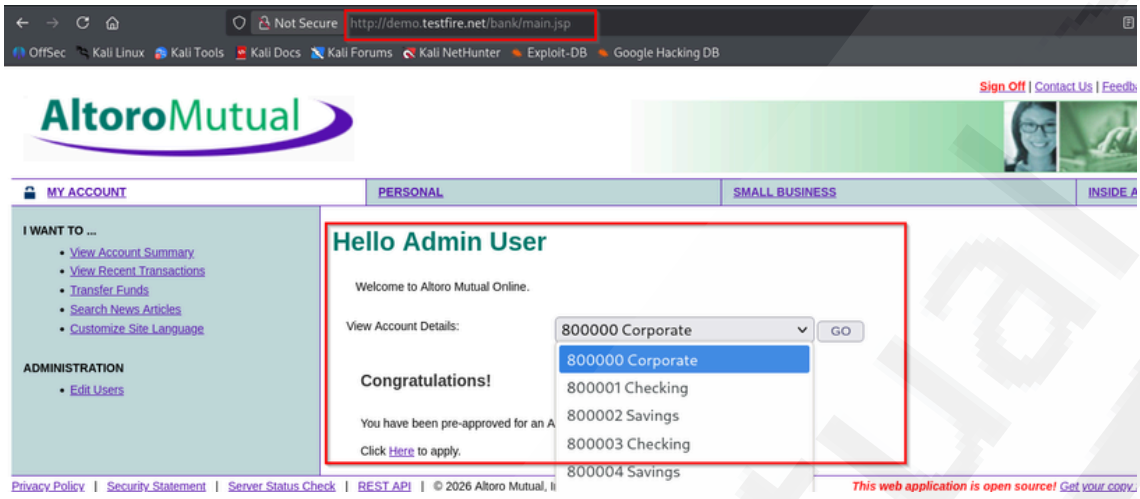
Was able to easily get through the login page of testfire.net, accessible at http://testfire.net/login.jsp by using the payload

Identify the Vulnerable Field: Enter a single quote ' in the Username field and submit the form with any password.



Result: If the application returns an SQL error message (e.g., "You have an error in your SQL syntax..."), it confirms SQLi.





Recommendations:

- Most instances of sql injection can be prevented by using parameterized queries (also known as prepared statements) instead of string concatenation within the query.
- Parameterized queries can be used for any situation where untrusted input appears as data within the query, including the WHERE clause and values in an INSERT or UPDATE statement. They can't be used to handle untrusted input in other parts of the query, such as table or column names, or the ORDER BY clause.
- For a parameterized query to be effective in preventing SQL injection, the string that is used in the query must always be a hard-coded constant, and must never contain any variable data from any origin.
- Do not be tempted to decide case-by-case whether an item of data is trusted, and continue using string concatenation within the query for cases that are considered safe. It is all too easy to make mistakes about the possible origin of data, or for changes in other code to violate assumptions about what data is tainted.

Brute Force

A successful brute-force attack leads to unauthorized access, system compromise	
Severity	MEDIUM
Affected URL	http://demo.testfire.net/login.jsp
Risk Score	6.5/10
CWE	Improper Restriction of Excessive Authentication Attempts
Labels	OWASP Top 10: A07:2021, OAT-009, ATT&CK Matrix: T1110, CAPEC-16

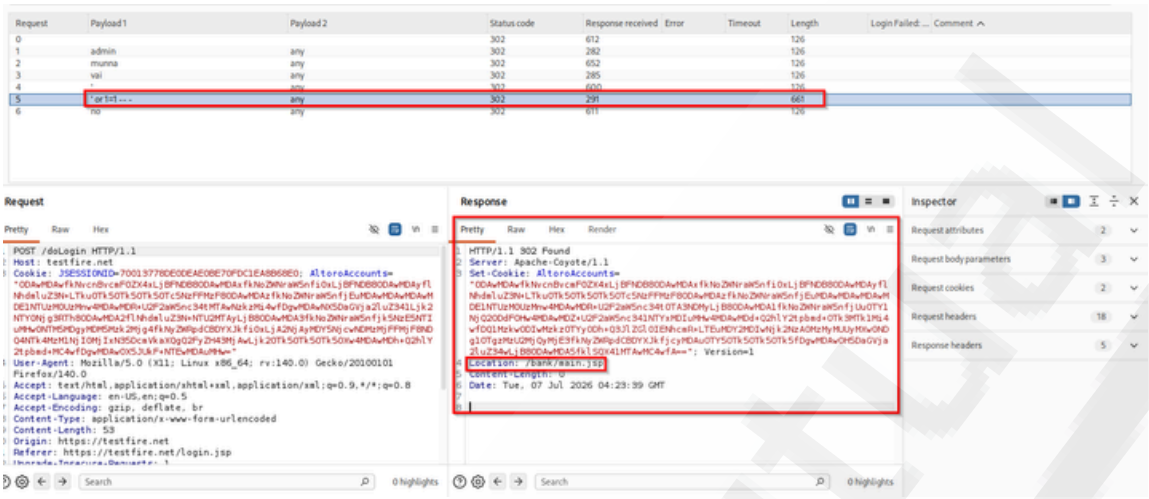
Description:

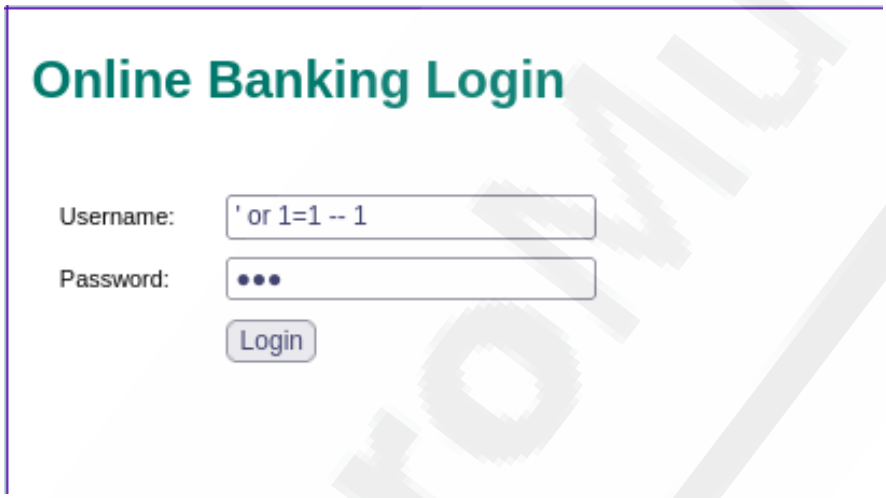
A brute-force attack consists of an attacker submitting many passwords or passphrases with the hope of eventually guessing correctly. The attacker systematically checks all possible passwords and passphrases until the correct one is found.

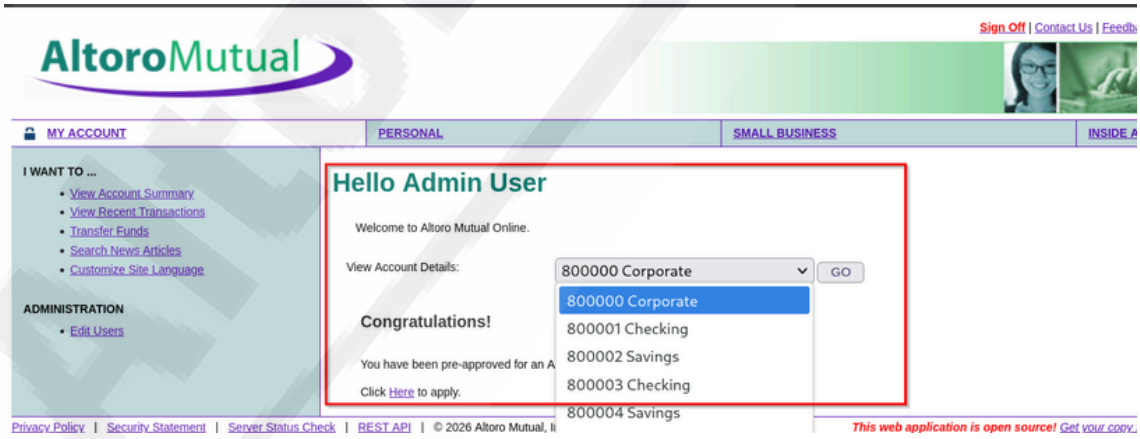
Impact:

- Attackers guess the website's user name and password by repeatedly attempting to conduct form-based login using user name and password in the dictionary.
- User name and password enumeration threatens websites that have no restriction on failed login attempts.

Proof of Concept (POC):







Recommendations:

1. Restrict login failures to lock the account when certain amount of login failures has reached.
   - The user-registered password must meet the intended complexity requirements. For example, the password must be a combination of no less than 8 characters of the following: lowercase letters, uppercase letters, digits, and special characters ~!@#$%^&*()-+_. For users with unqualified passwords, the registration should not be allowed.

2. Increase the complexity of the password.
   - Users are advised to change the default passwords occurring during installation to make them satisfy the preceding requirements.

Absence of Anti-CSRF Tokens

Attackers trick authenticated users into executing unauthorized	
Severity	MEDIUM
Affected URL	http://demo.testfire.net/login.jsp http://demo.testfire.net/feedback.jsp http://demo.testfire.net/subcribe.jsp
Risk Score	6.5/10
CWE	Improper Input Validation
Labels	OWASP Top 10: A01:2021, CAPEC-62, ATT&CK Matrix: T1566

Description:

No Anti-CSRF tokens were found in a HTML submission form.
A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf.

CSRF attacks are effective in a number of situations, including:

- The victim has an active session on the target site.
- The victim is authenticated via HTTP auth on the target site.
- The victim is on the same local network as the target site.

CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.

Impact:

- **Unauthorized Transactions:** An attacker can craft a hidden form that submits a fund transfer from the victim's account to an attacker-controlled account. If the victim is authenticated and visits a malicious page, the transfer executes silently.
- **Data Integrity Loss:** Profile details, contact information, security questions, or notification preferences can be modified without the user's intent.

- **Account Takeover:** Password change endpoints without CSRF protection allow an attacker to submit a new password on behalf of the victim — the victim's account is then fully compromised.
- **Privilege Escalation:** Admin-level actions (user creation, role modification, system configuration) can be forged if an admin user visits a crafted page, granting the attacker elevated access.
- **Session Persistence Manipulation:** Attackers can extend or modify session parameters, maintain persistent access, or disable logout functionality.
- **Chained Exploitation:** CSRF can be combined with other vulnerabilities (XSS, Clickjacking, HTML Injection) to increase severity — e.g., XSS to steal CSRF tokens is pointless if tokens are missing entirely.

Proof of Concept (POC):

uri	https://tesfire.net/login.jsp
method	GET
evidence	<form action="doLogin" method="post" name="login" id="login" onsubmit="return (confirminput(login));">
otherinfo	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token]



For Example

```
<form action="/transfer.do" method="post">

  <input type="hidden" name="CSRFToken"
value="OWY4NmQwODE4ODRjN2Q2NTlhMmZlYWEwYzU1YWQwMTVhM2JmNG
YxYjJiMGI4MjJjZDE1ZDZMGYwMGEwOA==">  [...]

</form>
```

uri	https://tesfire.net/feedback.jsp
method	GET
evidence	<form name="cmt" method="post" action="sendFeedback">
otherinfo	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token]



Recommendations:

- **Architecture and Design**
  - Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.
  - For example, use anti-CSRF packages such as the OWASP CSRFGuard.
- **Implementation**
  - Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script.
- **Architecture and Design**
  - Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330).
  - Note that this can be bypassed using XSS.
    Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation.
  - Note that this can be bypassed using XSS.
    Use the ESAPI Session Management control. This control includes a component for CSRF. Do not use the GET method for any request that triggers a state change.
- **Implementation**
  - Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.



Continuing...